CHAPTER THREE

RESEARCH METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

This chapter explains the methodology adopted for the design and implementation of the Multi-Tenant SaaS Platform for Business Inventory Management Using Microservices Architecture. It describes the research and requirements-gathering approach, analysis of requirements, proposed system architecture, design of the four microservices, tenant data-isolation strategy, communication mechanisms, development tools, implementation process and evaluation methods. The decisions presented here follow the approved proposal and the scope established in Chapters One and Two.

The project is an applied software development study. Its output is a working browser-based inventory management platform in which multiple independent businesses can register and manage stock and sales through one shared deployment. Each business is treated as a tenant. The system shares application infrastructure while keeping tenant records separate through a shared PostgreSQL database with a separate schema for each tenant.

3.2 Research and Development Approach

The study adopts an applied research and design-and-implementation approach. It identifies inventory management problems experienced by small and medium-sized businesses, translates those problems into system requirements, designs a suitable architecture, implements the system and evaluates it against the objectives stated in Chapter One. This approach is appropriate because the project's contribution is the construction and controlled evaluation of a working system, not only a discussion of multi-tenancy and microservices.

A combination of qualitative requirements gathering and software engineering methods is used. Interviews and document analysis provide information about business processes and required inventory records. The literature review provides architectural guidance on SaaS, multi-tenancy, data isolation and microservices. The software is developed incrementally using the Agile Scrum model specified in the proposal.

The interviews involve a small number of stock managers and small-business owners in Ede and Osogbo, as stated in Chapter One. Their responses are used to identify practical requirements and are not treated as a statistically representative survey of all Nigerian SMEs.

3.3 Requirements-Gathering Methods

3.3.1 Literature Review

Relevant journal articles, conference papers, books and technical documentation on SaaS, multi-tenant architecture, microservices, inventory management and tenant security are reviewed. Kabbedijk et al. (2015) and Guo et al. (2007) inform tenant identification and isolation; Chong, Carraro and Wolter (2006) inform the choice of a shared database with separate schemas; Newman (2021) informs service decomposition and service boundaries; and Batista et al. (2024) inform asynchronous messaging for tenant workloads.

3.3.2 Structured Interviews

Structured interviews are conducted with a small number of stock managers and small-business owners in Ede and Osogbo, Osun State. Questions focus on current stock-recording methods, difficulties encountered when goods are received or sold, how shortages are identified, how reorder decisions are made and which reports are useful. Responses are grouped into common operational needs. These needs guide the inventory dashboard, stock-entry process, sales recording and low-stock notification functions.

3.3.3 Document Analysis

Sample stock sheets, sales registers and inventory templates are examined to identify data fields and business rules used in ordinary small-business operations. The analysis determines the information represented in the system, including product descriptions, stock quantities, reorder levels, sales transactions and dates. This ensures that the data model reflects familiar business records.

Information from interviews and document analysis is compared with requirements identified in the literature. Features outside the approved scope are not added merely because they are common in larger commercial inventory platforms.

The requirements are translated into functional and non-functional requirements in Sections 3.6 and 3.7. Each requirement is then associated with the service or system component responsible for satisfying it. This provides traceability from the identified

business needs to the proposed design and the tests described in Section 3.14. The actual number and profile of interview participants, together with the interview instrument, should be stated accurately in the final report and supporting appendix; no participant count is assumed here.

3.4 Analysis of the Existing System

The existing practice identified in Chapters One and Two is that many small businesses manage stock through handwritten records, spreadsheets or basic desktop applications. These methods make it difficult to maintain accurate and timely stock information, especially when sales and stock movements occur frequently. Records may be duplicated, updates delayed, and business owners may lack a reliable view of current stock when away from the business premises.

The reviewed Nigerian inventory systems demonstrate that web applications can improve stock recording and reporting for an individual organisation (Erameh & Odoh, 2021; Chukwumuanya et al., 2024). However, the systems reviewed in Chapter Two are designed for a single organisation and do not address secure sharing of one software deployment among unrelated businesses. Commercial SaaS platforms provide hosted inventory functionality, but Chapter One identifies affordability and local-context concerns for Nigerian SMEs.

The proposed system therefore addresses two related needs: computerised stock and sales management through a browser, and shared use of one platform by multiple businesses without allowing them to access one another’s records.

3.5 Analysis of the Proposed System

The proposed system is a web-based, multi-tenant SaaS inventory platform. A business registers and is assigned a unique tenant identity. The tenant identity is associated with the authenticated user and establishes the business context of each request. Each tenant receives a separate PostgreSQL schema within one shared database. Application code and infrastructure are shared, while tenant-owned records are separated by schema.

The system provides the following core capabilities:

Business registration and tenant onboarding, including creation of a tenant record and isolated data schema.

User authentication through the Authentication and Tenant Management Service.

Product and stock management, including creating products, updating stock quantities and monitoring reorder levels.

Order and sales recording for the authenticated tenant.

Low-stock event generation and notification through the Notification Service.

Basic stock and sales summaries through the React.js web interface.

Tenant-aware access through REST endpoints exposed by the individual microservices, with each service validating authentication and resolving tenant identity.

Controlled evaluation of response time, tenant scalability and cross-tenant data isolation.

The system excludes native mobile applications, payment gateway integration, subscription billing, multi-currency support, advanced demand forecasting, machine-learning analytics and commercial production deployment. These exclusions preserve the boundaries stated in Chapter One.

3.6 Functional Requirements

The functional requirements describe the operations the completed system is expected to perform.

FR1: The system shall allow a business to register and create a tenant account (Authentication and Tenant Management Service).

FR2: The system shall authenticate registered users and issue a signed access token (Authentication and Tenant Management Service).

FR3: The system shall associate authenticated users with their tenant identity (Authentication and Tenant Management Service).

FR4: The system shall create a separate PostgreSQL schema for each newly registered tenant (tenant provisioning logic).

FR5: The system shall allow an authenticated tenant user to create, view and update product records (Inventory Service).

FR6: The system shall allow an authenticated tenant user to record stock changes and view current stock quantities (Inventory Service).

FR7: The system shall allow an authenticated tenant user to record and view

sales/orders (Order and Sales Service).

FR8: The system shall update the relevant stock quantity when a sale is processed, using a defined service interaction (Order and Sales / Inventory Services).

FR9: The system shall identify stock levels that reach or fall below a product's reorder level (Inventory Service).

FR10: The system shall publish a low-stock event for the Notification Service to process (Inventory Service / RabbitMQ).

FR11: The system shall generate a tenant-scoped notification using the channel actually implemented (Notification Service).

FR12: The system shall display basic stock and sales summaries for the authenticated tenant (React client and relevant services).

FR13: Each protected microservice shall validate incoming access tokens and derive the authenticated user's tenant identity from verified token claims (relevant microservice).

FR14: The system shall prevent a tenant user from viewing or modifying another tenant's records through service-level authorisation and tenant-aware data access (microservices and data-access layer).

The notification delivery channel must match the implementation. If email delivery is not implemented, the report should describe the feature as an in-application notification rather than claim an email capability.

3.7 Non-Functional Requirements

Security: Passwords shall be stored using an appropriate password-hashing algorithm. Protected endpoints shall require a valid token. Tenant context shall be derived from verified authentication information, not trusted directly from a client-supplied tenant identifier.

Tenant isolation: Each tenant shall have a separate schema. Every tenant-scoped request shall be routed to the correct schema by the service data-access layer. Cross-tenant access attempts shall be denied and tested.

Performance: API response time shall be evaluated under simulated concurrent workloads. Results shall be measured and reported rather than assumed.

Scalability: The system shall be tested with increasing numbers of simulated tenants and concurrent users to identify the load it supports within defined response-time and error-rate criteria.

Maintainability: The application shall be divided into four services with clearly defined responsibilities and documented REST interfaces or message contracts.

Resilience: Failure in asynchronous notification processing should not unnecessarily block ordinary stock and sales requests. Message handling shall include acknowledgement and controlled retry behaviour where implemented.

Usability: The React.js interface shall provide clear forms and views for product, stock, sales and summary operations through a modern browser.

Portability: Docker and Docker Compose shall provide a repeatable local development and demonstration environment.

3.8 System Architecture

The proposed architecture is a tenant-aware microservices architecture organised into four logical layers: Client Layer, Microservices Layer, Data Layer and Messaging Layer. The React.js client sends HTTPS requests directly to the REST endpoints exposed by the relevant microservice. Each protected microservice validates the access token, derives tenant identity from verified token claims and performs its own business operations. Services access only the tables they own within the schema resolved for the authenticated tenant. RabbitMQ carries asynchronous events between services.

3.8.1 Client Layer

The client layer is implemented with React.js. It provides the browser interface through which a tenant user signs in and performs inventory operations. The main interface areas are the inventory dashboard, product and stock-entry forms, sales log and basic reporting views. The client does not connect directly to PostgreSQL or RabbitMQ. It sends each request over HTTPS to the REST endpoint of the service responsible for that operation.

3.8.2 Authentication and Tenant Management Service

This service manages business registration, tenant records, user authentication and the association between a user and a tenant. It maintains the platform-level tenant registry in a central PostgreSQL schema, such as a platform schema. The exact schema name is an implementation detail and must be consistent in the migrations and source code. On

registration, the service creates the tenant record and provisions the tenant's separate schema. It creates or associates the initial user account with that tenant.

After successful authentication, the service issues a signed JWT access token containing the user identifier and tenant identifier, together with standard metadata such as expiry. Other services trust the tenant context only after verifying the token signature and expiry. Passwords are never stored in plain text.

3.8.3 Inventory Service

The Inventory Service manages product information and stock quantities. Its responsibilities include creating product records, retrieving product details, updating product information, recording stock changes and checking stock against reorder levels. When a stock operation causes the quantity to reach or fall below the configured reorder level, the service publishes a low-stock event to RabbitMQ. The event contains the tenant identity and the minimum information required for the notification service.

3.8.4 Order and Sales Service

The Order and Sales Service records sales/orders for the tenant and provides sales history and summaries. It validates the requested sale and coordinates the required stock update through the Inventory Service API rather than writing directly to inventory tables. This keeps inventory ownership within the Inventory Service. The platform records sales information; it does not process customer payments or integrate a payment gateway.

3.8.5 Notification Service

The Notification Service consumes relevant events from RabbitMQ and generates low-stock notifications for the tenant concerned. It uses the tenant identity carried in the event to establish the correct tenant context before storing or delivering a notification. The service validates the event and does not treat an unverified tenant identifier in a message as sufficient authorisation.

3.8.6 Data Layer and Service Data Ownership

The project uses one PostgreSQL database with a separate schema for each tenant, as required by Chapters One and Two. A central platform schema stores global records needed to identify tenants and users. Each tenant schema contains that tenant's business data. Within each tenant schema, tables are assigned to the service that owns the corresponding business capability. The Inventory Service owns product and stock tables;

the Order and Sales Service owns order and sales tables; and the Notification Service owns notification records if persistent notification history is implemented. The Authentication and Tenant Management Service owns central tenant and authentication records.

This ownership rule reconciles the shared-database design with the service autonomy principle discussed in Chapter Two. Services share the PostgreSQL server and tenant schema namespace, but they do not query or modify another service's tables directly. When one service needs an operation owned by another service, it uses that service's REST API or an agreed event. Database credentials and permissions should follow least privilege where the implementation environment permits.

A separate schema is a namespace boundary, not an automatic security guarantee. The application must select the schema only after validating tenant identity, avoid unsafe dynamic SQL and prevent untrusted input from controlling schema names. PostgreSQL privileges and carefully managed schema resolution provide additional protection. OWASP's multi-tenant guidance highlights disciplined grants and search_path handling for schema-based isolation (OWASP Foundation, 2026).

3.8.7 Messaging Layer

RabbitMQ is selected because it is identified in the proposal and supports asynchronous communication. The principal event is the low-stock event. The Inventory Service publishes it after detecting a stock threshold condition. The Notification Service consumes it and performs the notification operation. This prevents the inventory request from waiting for all notification processing to finish.

A message should contain an event identifier, event type, tenant identifier, relevant product identifier and event timestamp. The consumer validates required fields and acknowledges a message only after successful processing. Failed messages may be retried a limited number of times; messages that repeatedly fail should be logged for investigation rather than retried indefinitely. RabbitMQ documents publisher confirms and consumer acknowledgements as mechanisms supporting reliable message transfer (RabbitMQ, 2026). Only reliability controls that are implemented and tested should be claimed.

3.8.8 Containerisation and Deployment Arrangement

Docker packages each microservice into separate containers. The React.js client, PostgreSQL database and RabbitMQ broker are also run as services in the Docker Compose development environment. Compose defines services, environment variables,

ports, networks and persistent volumes required to run the platform locally. This provides a repeatable development, test and demonstration environment; it does not mean that the project has been deployed as a production commercial SaaS platform.

3.9 Tenant Onboarding and Data-Isolation Process

A business submits its registration details through the React.js client.

The registration request is sent directly to the Authentication and Tenant Management Service through its REST endpoint.

The service validates the details and creates a tenant record in the central platform schema.

The service generates a safe schema identifier from a server-controlled tenant identifier. It does not use a raw business name supplied by the user as an SQL schema identifier.

The service provisions the tenant schema and applies the required migrations to create its tables.

The service creates or associates the initial user account with the tenant and completes registration.

On subsequent login, the service authenticates the user and issues a signed JWT containing the tenant identity.

For each protected request, the destination microservice verifies the JWT, derives the user and tenant context from its validated claims, and passes that trusted context to its data-access layer.

The service data-access layer resolves the tenant schema from verified context and performs operations only within that tenant schema.

If schema creation or migration fails, the tenant must not be treated as fully provisioned. The service should return a controlled error and avoid leaving an active tenant record that points to an incomplete schema. Isolation tests create at least two tenants with distinguishable records, then attempt to access one tenant's records while authenticated as the other. The expected result is denial or an empty tenant-scoped result, never disclosure or modification of the other tenant's data.

Isolation tests should use the same request path and database role as the application.

Testing only with a database administrator account can conceal weaknesses in application permissions (OWASP Foundation, 2026).

3.10 Database Design

The database contains a central schema for platform-level records and one schema for each registered tenant. The central schema stores tenant identity and the minimum user/account information required to authenticate and resolve tenant membership. Tenant schemas store business records.

Central platform schema: tenant records, user or identity references, tenant status and schema mapping. Owned by Authentication and Tenant Management.

Tenant schema – inventory: product records and stock movements or stock adjustments. Owned by Inventory Service.

Tenant schema – sales: sales orders and sale/order items. Owned by Order and Sales Service.

Tenant schema – notifications: notification records, if notification history is persisted. Owned by Notification Service.

These are logical entity examples, not a claim that these exact physical table names already exist in the source code. Final table names, columns, constraints and indexes must match the database migrations used in the implementation.

The labels inventory, sales and notifications describe groups of service-owned tables within each tenant's schema; they do not represent three separate schemas for each tenant. For example, one tenant schema may contain product and stock tables owned by Inventory Service, sales tables owned by Order and Sales Service, and notification tables owned by Notification Service where notification history is persisted. Each service accesses only its own tables.

Primary keys uniquely identify records. Foreign keys are used for relationships within a service's owned tables. Constraints prevent invalid values, such as negative quantities where disallowed by the business rules. Indexes should be created on frequently queried fields, including product identifiers, sale dates and relationship keys. Each tenant schema is created from the same migration structure so that all tenants have a consistent data model.

A sale that affects stock is coordinated through the Inventory Service API. The Order and Sales Service must not directly update the Inventory Service's stock table. The sale

operation should use a unique sale reference so that retries do not deduct stock more than once. The Order and Sales Service first records the sale as pending, requests stock validation and deduction from the Inventory Service, and marks the sale as completed only after the stock operation succeeds. If the stock operation fails, the sale is marked failed and no completed sale is recorded. If stock is deducted but the sales record cannot be completed because of a subsequent failure, the system should issue a compensating stock adjustment using the original sale reference and record the outcome for reconciliation. This is a bounded failure-handling approach for the project; a full distributed transaction protocol is outside the required scope.

3.11 System Process Flow

3.11.1 User Authentication

A user opens the web application and submits login credentials. The React.js client sends the authentication request directly to the Authentication and Tenant Management Service. The service verifies the credentials, retrieves the user's tenant association and returns a signed JWT. The client includes the token as a bearer token in subsequent protected requests. Each destination microservice validates the token before processing a protected request.

When the Order and Sales Service calls the Inventory Service on behalf of an authenticated user, it forwards the user's bearer token over the service-to-service connection. The Inventory Service independently verifies the token signature, expiry and relevant permissions before performing the stock operation. It derives the tenant context from the verified claims; it does not rely on a tenant identifier supplied separately in the request body or an untrusted header. The deployed environment should protect service traffic with HTTPS or an appropriately restricted private network.

3.11.2 Product and Stock Management

An authenticated tenant user submits a product or stock operation to the Inventory Service REST endpoint. The Inventory Service validates the JWT, resolves the tenant schema from the verified tenant context, validates the data and performs the operation on its own tables. If the resulting stock quantity reaches the reorder threshold, the service publishes a low-stock event to RabbitMQ.

3.11.3 Sales Recording

An authenticated user submits a sale through the React.js interface to the Order and Sales Service REST endpoint. The service validates the JWT and sale details, then requests the Inventory Service through its REST interface to check and update the relevant stock. If the stock operation succeeds, the sales service records the sale and returns the result to the client. The platform records the sale but does not charge a customer or process a payment.

3.11.4 Low-Stock Notification

The Inventory Service publishes a low-stock event to RabbitMQ. The Notification Service receives the event, validates its content and tenant context, and creates or delivers the corresponding notification. Because this operation is asynchronous, the stock operation does not need to wait for notification delivery to complete.

3.12 Software Development Tools and Technologies

Node.js: JavaScript runtime used to execute backend services.

NestJS: Backend framework used to structure the four microservices with modules, controllers, providers and validation.

React.js: Frontend library used to build the browser-based dashboard, forms, sales log and basic reports.

PostgreSQL: Relational database used for central platform records and tenant-specific business data in separate schemas.

RabbitMQ: Message broker used for asynchronous events such as low-stock notifications.

Docker: Container technology used to package independently deployable backend components.

Docker Compose: Tool used to define and run the local multi-container development and test environment.

Postman: Used to send requests to REST endpoints and verify responses and error handling.

Jest: Used for unit and integration tests of backend components and service behaviour.

k6 (recommended): Used to generate simulated concurrent requests and measure response-time and error-rate behaviour during load testing.

Git and GitHub: Used for source-code version control, change tracking and project backup.

The backend uses NestJS across the four microservices to provide a consistent modular structure. Each service exposes its own REST endpoints and validates protected requests using the shared JWT verification rules. No separate API Gateway application is included in the project scope.

k6 is recommended because it supports scripted virtual users and measurable thresholds for response duration and failed requests. Grafana's documentation describes thresholds as pass/fail criteria for metrics such as response duration and request failure rate (Grafana Labs, 2026). k6 should only be reported as used if the tests are actually executed.

3.13 Software Development Methodology

The project adopts the Agile Scrum model stated in the approved proposal. Development is organised into short iterations, with each iteration producing a working increment. The cycle consists of the following activities:

Requirements: select the feature or service for the sprint and confirm its functional and non-functional requirements.

Design: define the service responsibility, API contract, data model changes and tenant context required.

Develop: implement the feature in the relevant NestJS service or React.js client and commit changes to Git.

Test: run unit tests, API tests and integration checks for the new or modified functionality.

Deploy: build or update the relevant Docker image and run the service in the Docker Compose environment.

Review: demonstrate the working increment, record defects and feedback, and refine the backlog for the next iteration.

A practical implementation order is to establish the database and tenant model first, then implement authentication and tenant provisioning, followed by Inventory, Order and Sales, Notification, and the React.js client. Integration and evaluation are then carried out across the complete system. This order reduces the risk of implementing business services before tenant identity and isolation mechanisms are available.

3.14 System Testing and Evaluation Plan

Evaluation is aligned directly with the three metrics stated in the proposal and Chapter One: API response time under simulated load, tenant scalability and data-isolation integrity. Tests are performed in a controlled development/test environment. Actual results will be presented in Chapter Four; no performance result is assumed in this chapter.

3.14.1 Functional and API Testing

Postman is used to verify successful registration and login, rejection of invalid credentials, product creation and retrieval, stock updates, sale recording, low-stock event handling and basic report retrieval. Negative tests include missing, invalid or expired tokens, invalid input values and attempts to access another tenant's resources. Jest is used for automated unit and integration tests of service logic, validation, tenant-context resolution and event handling.

3.14.2 API Response-Time Evaluation

Response time is measured for representative operations, particularly product retrieval, stock update and sale creation. A load-testing script sends requests under controlled levels of simulated concurrency. For each test level, average response time, median and selected percentiles such as the 95th percentile, together with request failure rate, are recorded. The test environment, request mix, duration, virtual-user count and warm-up conditions are documented.

A practical project target may be defined before testing, for example a 95th-percentile response time below 1,000 milliseconds for selected core endpoints and an error rate below 1 percent under the stated test load. These are proposed acceptance targets, not results or guaranteed service-level objectives. If the system does not meet them, the measured outcome must be reported honestly.

3.14.3 Tenant Scalability Evaluation

The platform is tested with increasing tenant accounts and simulated concurrent users. A practical sequence for a student test environment is to begin with two tenants, then increase the test dataset and active simulated users in controlled steps, for example 5, 10, 20 and 30 tenants where machine resources permit. The same workload profile is applied at each step. Response time and error rate are compared to identify the highest tested configuration that remains within the acceptance criteria.

The number of tenant accounts must be distinguished from the number of simultaneous virtual users. A test with 20 tenant accounts and 50 virtual users does not mean 50 tenants were active. Both figures should be stated in Chapter Four.

3.14.4 Data-Isolation Integrity Evaluation

At least two tenant accounts are created and populated with distinguishable product and sales records. A user authenticated as Tenant A attempts to retrieve or update records belonging to Tenant B by using a known record identifier or manipulating request parameters. The tests are repeated in the opposite direction. The expected result is that the operation is rejected or returns no record, and that no other tenant's data is changed. Tests also verify that schema resolution uses the validated token rather than an untrusted client-supplied value.

Test cases and outcomes are recorded in a matrix containing test identifier, authenticated tenant, attempted operation, expected result, actual result and pass/fail status. These results provide evidence for the data-isolation objective.

3.14.5 Evaluation Summary

API response time: run representative API requests at defined concurrency levels; report average and percentile response times, request count and error rate.

Tenant scalability: increase tenant dataset and concurrent virtual users in controlled stages; report the highest tested configuration meeting the stated criteria.

Data isolation integrity: attempt cross-tenant reads and writes using authenticated accounts from at least two tenants; report test cases, expected and actual responses, and pass/fail results.

3.15 Security and Reliability Considerations

Security is a core architectural requirement because the platform stores records belonging to independent businesses in shared infrastructure. The design includes the following controls:

Authentication is required for protected tenant operations, and access tokens are signed and time-limited.

Tenant identity is derived from validated token claims. Client-supplied tenant identifiers are not accepted as proof of authority.

Each tenant has a separate PostgreSQL schema, and the application resolves the schema through trusted server-side logic.

Services access only the tables they own. Cross-service operations use REST APIs or RabbitMQ events rather than direct access to another service's tables.

Input validation is applied to API requests, and database operations use parameterised queries or safe ORM methods.

Passwords are stored as hashes using an appropriate password-hashing function, not as plain text.

Internal messages are validated, tenant context is carried with tenant-scoped events, and consumers acknowledge messages only after successful processing.

Errors returned to clients do not expose passwords, access tokens, database credentials or internal stack traces.

The OWASP Multi-Tenant Security Cheat Sheet (2026) recommends deriving tenant context from server-verified identity, enforcing isolation at the data-access boundary, controlling schema privileges and testing cross-tenant denial. These recommendations support the separate-schema design. A separate schema alone is not sufficient; the application must consistently enforce the tenant boundary.

3.16 Ethical Considerations and Study Limitations

Interview participants are approached for requirements-gathering purposes, and information provided is used to understand inventory workflows. Business-sensitive information should not be reproduced in the report without permission. Sample stock sheets and sales registers should be anonymised where they contain identifiable business or customer information.

The study is subject to the limitations stated in Chapter One. The interview sample is small and limited to selected businesses in Ede and Osogbo; it is not statistically representative of all Nigerian SMEs. Performance tests use simulated workloads in a controlled environment rather than live production traffic. Available cloud resources and student credits constrain the scale and duration of testing. The project focuses on core inventory, sales and notification functions rather than every feature found in commercial inventory platforms.

The system is a development and test implementation. Results should be interpreted as

evidence of architecture behaviour under the stated test conditions, not proof of production readiness or guaranteed performance for a large commercial customer base.

3.17 Chapter Summary

This chapter has presented the methodology and system design for the Multi-Tenant SaaS Platform for Business Inventory Management Using Microservices Architecture. It described the applied design-and-implementation approach and the use of literature review, structured interviews and document analysis to establish requirements. It analysed the limitations of manual, spreadsheet-based and single-tenant inventory practices and defined the functional and non-functional requirements. The chapter specified a layered architecture consisting of a React.js client, four NestJS microservices, a shared PostgreSQL database with a separate schema for each tenant, and RabbitMQ for asynchronous events. The choice of Docker, Docker Compose, Postman, Jest, Git and the recommended k6 load-testing tool was explained. Finally, it established the Scrum development process and testing plan for response time, tenant scalability and data-isolation integrity. Chapter Four will present implementation details, test execution and results obtained from the completed system.

REFERENCES

Afolayan, A., Plant, E., White, G. R. T., Jones, P., & Beynon-Davies, P. (2015). Information technology usage in SMEs in a developing economy. Strategic Change, 24(5), 483–498.

Apulu, I. (2012). Developing a framework for successful adoption and effective utilisation of ICT by SMEs in developing countries: A case study of Nigeria [Doctoral dissertation, University of Wolverhampton].

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. Communications of the ACM, 53(4), 50–58.

Batista, C., Cavalcante, W. B. R., Proença, B., Batista, T., & Morais, F. (2024). Managing asynchronous workloads in a multi-tenant microservice enterprise environment. Software: Practice and Experience, 54(2), 334–359.

Bezemer, C.-P., Zaidman, A., Platzbeecker, B., Hurkmans, T., & Hart, A. (2010). Enabling multi-tenancy: An industrial experience report. Proceedings of the 2010 IEEE International Conference on Software Maintenance, 1–8.

Chong, F., Carraro, G., & Wolter, R. (2006). Multi-tenant data architecture. Microsoft Corporation.

Chukwumuanya, O. E., Onwurah, U. O., & Ihueze, C. C. (2024). Development of a web-based inventory management system for small businesses. International Journal of Industrial and Production Engineering, 2(2), 53–67.

Erameh, K. B., & Odoh, B. I. (2021). Design and implementation of a web-based inventory control system using a small medium enterprise (SME) as a case study. NIPES Journal of Science and Technology Research, 3(3), 211–219.

Grafana Labs. (2026). Thresholds. Grafana k6 documentation. https://grafana.com/docs/k6/latest/using-k6/thresholds/ (Accessed 8 November 2026).

Guo, C. J., Sun, W., Huang, Y., Wang, Z. H., & Gao, B. (2007). A framework for native multi-tenancy application development and management. Proceedings of the 9th IEEE International Conference on E-Commerce Technology, 551–558.

Kabbedijk, J., Bezemer, C.-P., Jansen, S., & Zaidman, A. (2015). Defining multi-tenancy: A systematic mapping study on the academic and the industrial perspective. Journal of Systems and Software, 100, 139–148.

Kavis, M. J. (2014). Architecting the cloud: Design decisions for cloud computing service models (SaaS, PaaS, and IaaS). John Wiley & Sons.

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

OWASP Foundation. (2026). Multi-tenant security cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html (Accessed 8 November 2026).

RabbitMQ. (2026). Consumer acknowledgements and publisher confirms. https://www.rabbitmq.com/docs/confirms (Accessed 8 November 2026).